

PROCEDURAL ANIMATION

Aashish Kumar Bankala | VFX 350 | Project 3 | Spring 2026
05/26/2026 | Houdini version: 21.0.440

IMPORTANT STATISTICS

RENDER

- Renderer - Karma XPU, 28 samples
- Average render time - 30 sec/frame
- Image resolution - 1920x1080
- Number of lights - 2 lights (HDRI Domelight, area light)

SAMPLING

- Noise Value - 0.01
- Min/Max Rays - 1/9
- Diffuse - 1
- Reflection - 4
- Global Quality - 1

GEOMETRY COMPLEXITY

- Primitives: 82 (packed)
- Points: 101 (packed)

PROJECT DESCRIPTION

A procedural symbiote creature animation made in Houdini using VEX, VDB, Scatter nodes.

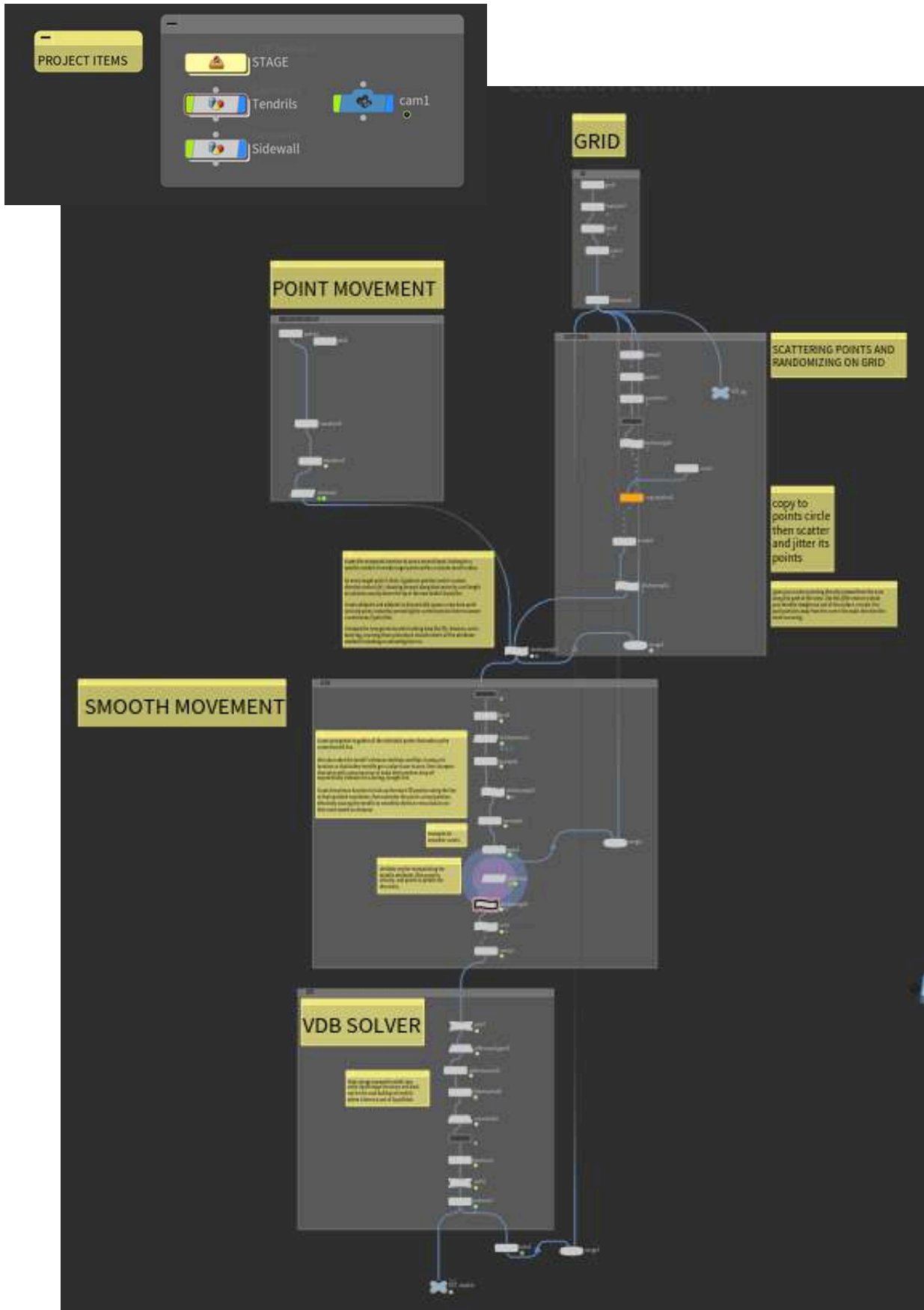
REFERENCE



RESULT



TECHNICAL GUIDE



GRID

SCATTERING POINTS AND RANDOMIZING ON GRID

copy to points circle then scatter and jitter its points

Randomize on grid

-

gives you a vector pointing directly outward from the core along the path of the noise. Use this `@dir` vector to shoot your tendrils straight out out of the surface, or scale it to gush particles away from the core in the exact direction the mesh is moving.

The screenshot shows a Houdini node network on the left and a VEX expression on the right. The node network includes a 'scatter4' node connected to an 'attribwangle2' node. The VEX expression is as follows:

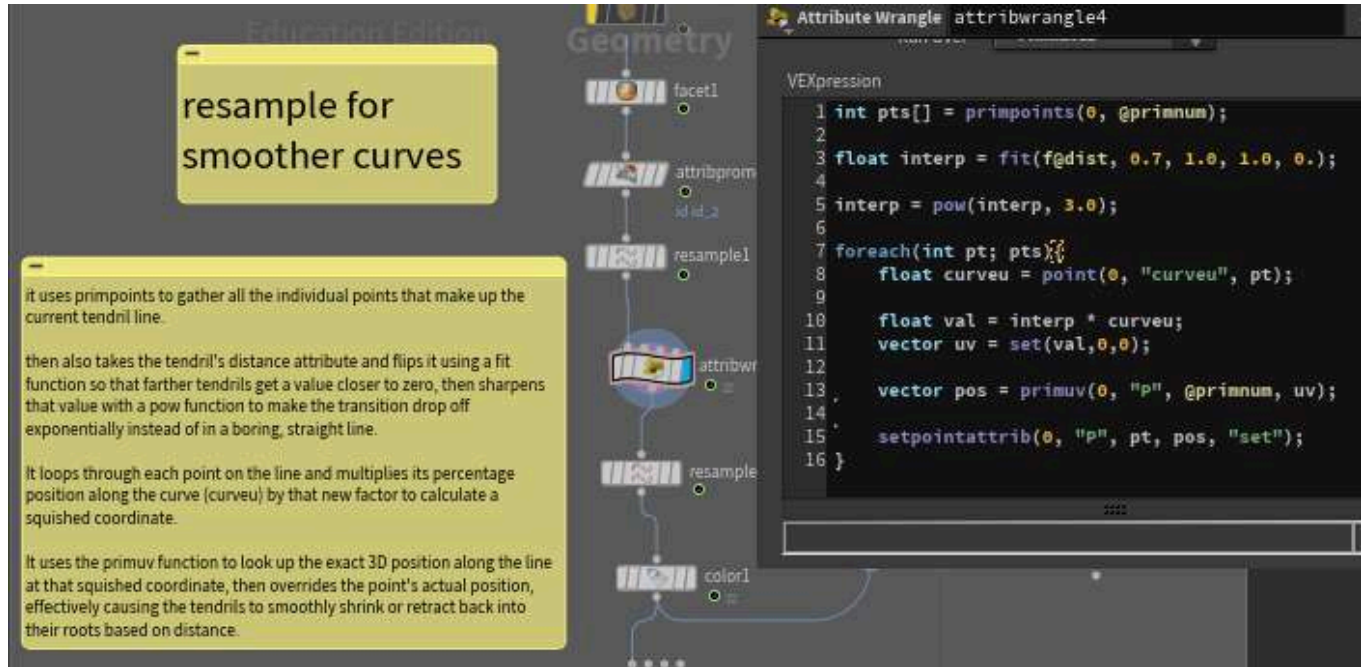
```
VEXexpression
1 i@id_2 = @ptnum +1;
2 v@dir = normalize(@p - v@rest_pos);
```

Below the VEX expression, a yellow tooltip box contains the text: "gives you a vector pointing directly outward from the surface".

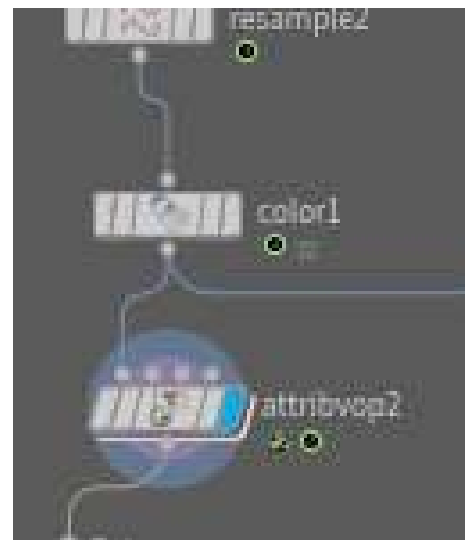
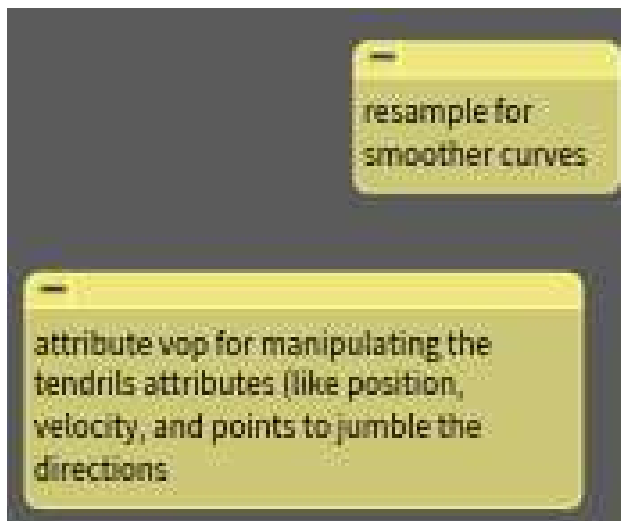
```

1 float rad = chf("tendrill_reach_radius");
2 int legs = chi("tendrill_strands_amt");
3 float length = chf("tendrill_strands_length");
4
5 int pts[] = nearpoints(1, @P, rad, legs);
6
7 foreach(int pt; pts){
8     vector pos = point(1, "p", pt);
9     float dist = length(@P - pos);
10    int id_2 = point(1, "id_2", pt);
11    int id = point(1, "id", pt);
12    vector dir = point(1, "dir", pt);
13    vector t_pos = pos + dir * length;
14
15    int new_pt = addpoint(0, pos);
16
17    int t_pt = addpoint(0, t_pos);
18
19    int prim = addprim(0, "polyline", @ptnum, new_pt, t_pt);
20
21    setpointattrib(0, "to_bevel", new_pt, 1, "set");
22    setprimattrib(0, "id", prim, id, "set");
23    setprimattrib(0, "id_2", prim, id_2, "set");
24    setprimattrib(0, "dist", prim, dist, "set");
25 }

```

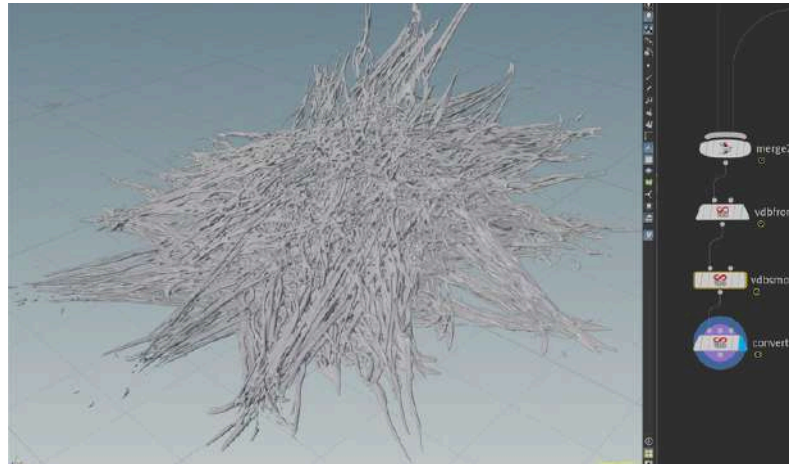
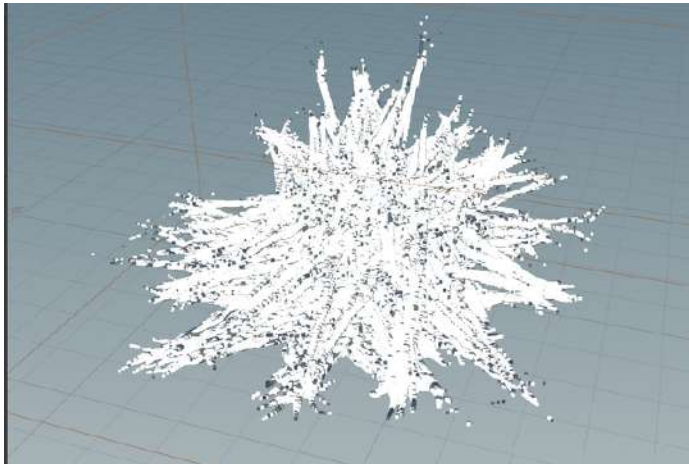
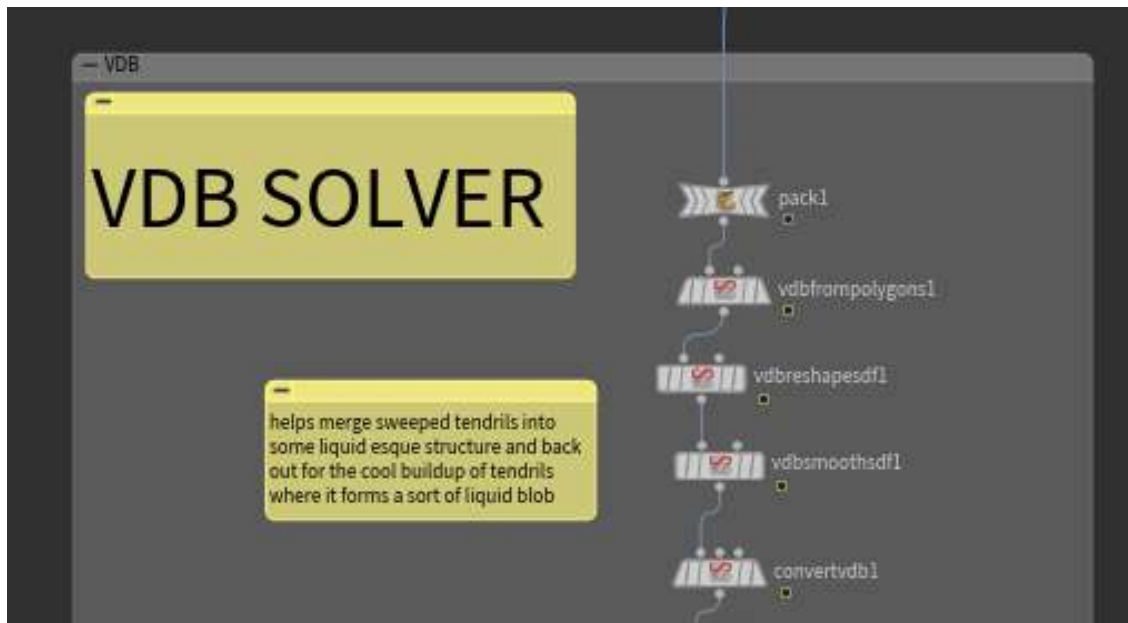



- Point Deform: Warps the stable tendril lines perfectly onto the noise animated base.

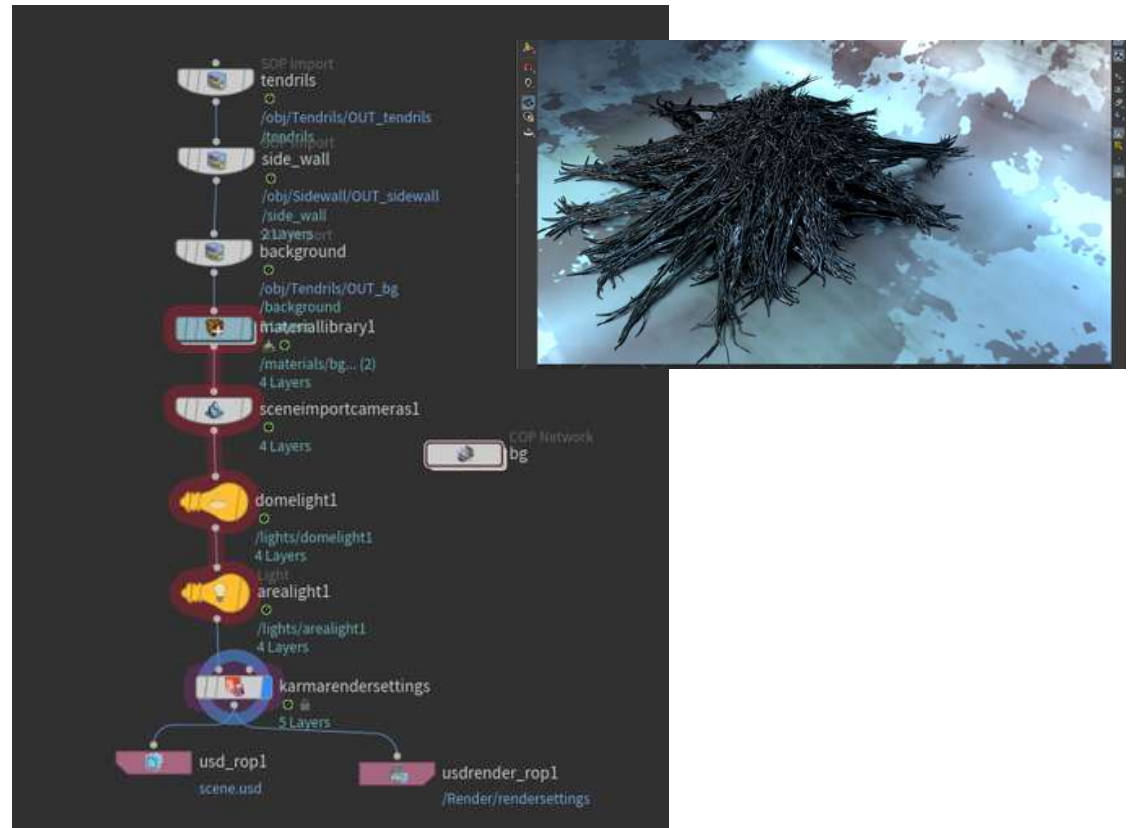


- Inside the node network, it maps a custom ramp or noise along that curve path to control the thickness, ensuring the strands taper down into sharp, thin needles at the ends.

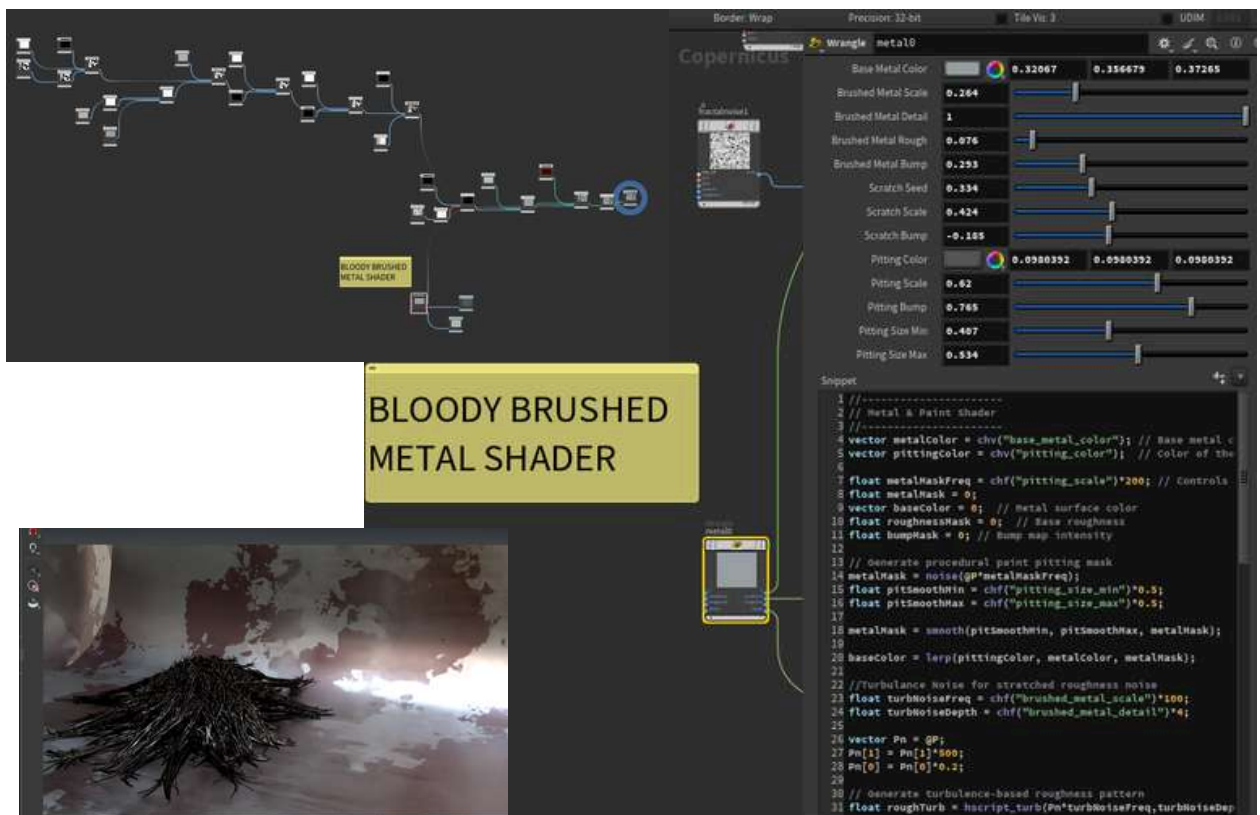
- VDB!!!: Sweep mesh lines into capped tubes, then SDF(Signed Distance Field)-melts them together, and uses Adaptivity to crush poly counts.



- LOPNET:



- Base bloody brushed metal shader:



FINAL RENDERS



CHALLENGES

- Anti-Aliased Flow Noise: (aaflownoise) Discovering enabling 3D Noise, it was 2D noise and there were a few glitches that way previously.
- Faceting: there's a checkbox for "unique points" that I couldn't figure out and it was also make the tendrils look weird.
- VDB Smooth: smoothed it a lot, had to tweak values and chache multiple times to see what looked right.
- Disappearing Tips in VDB: When converting the thin tendril geometry into VDBs, the VDB Smooth node kept erasing the fine tips and shrinking the mesh into blocky stubs. Mostly fixed this by increasing the offset value in VDB and then the sweep mesh thickness before smoothing.

BIBLIOGRAPY AND REFERENCES

- **Professor Deborah Fowler**
- www.deborahfowler.com
- **Marvel fandom**
- <https://marvel-movies.fandom.com/wiki/Category:Symbiotes>
- **Special thanks to the Houdini YouTube community**
- <https://www.youtube.com/watch?v=zH8F5CQerIE&list=PLmbKyINQ-noDyW4CdJM0DS40BlGqy57OC&index=1>
- https://www.youtube.com/watch?v=5C5NY_rbqEs&t=5s
- **Gemini AI**
- <https://gemini.google.com/app>